

CSA0624 - Design and Analysis of Algorithms

C02_AT1

NAME : Aryan ARUN

REG.NO : 192411157

Q1. Selection Sort — Analytical Problem

1. Problem Interpretation

We are given the unsorted dataset [64, 25, 12, 22, 11] and asked to arrange it in ascending order using the Brute Force technique known as Selection Sort. Brute Force here means the algorithm solves the problem by directly and exhaustively examining the data — repeatedly scanning the unsorted portion of the array to find the smallest element, rather than using any clever partitioning or divide-and-conquer strategy.

2. Why Selection Sort is a Suitable Brute Force Method

- It directly implements the brute-force idea: for each position, exhaustively scan the remaining unsorted elements to locate the minimum — no shortcuts or precomputation.
- It is simple to trace by hand, which makes it ideal for step-by-step analytical demonstration.
- It performs a predictable, fixed number of comparisons regardless of the initial arrangement, which makes its complexity easy to analyze rigorously.
- It minimizes the number of swaps (at most $n-1$), which is useful when the cost of swapping/writing is high.

3. Step-by-Step Execution

Selection Sort works by dividing the array into a sorted part (left) and an unsorted part (right). In each pass, the minimum element of the unsorted part is found and swapped into its correct position.

Pass	Unsorted Segment Examined	Comparisons	Minimum Found	Array After Pass
1	[64, 25, 12, 22, 11]	4	11 (index 4)	[11, 25, 12, 22, 64]
2	[25, 12, 22, 64]	3	12 (index 2)	[11, 12, 25, 22, 64]
3	[25, 22, 64]	2	22 (index 3)	[11, 12, 22, 25, 64]
4	[25, 64]	1	25 (already placed)	[11, 12, 22, 25, 64]

Detailed trace of Pass 1: starting with the assumed minimum 64 at index 0, compare it against 25 ($25 < 64$, new $\text{min}=25$), then 12 ($12 < 25$, new $\text{min}=12$), then 22 ($22 > 12$, no change), then 11 ($11 < 12$, new $\text{min}=11$). The minimum 11 is at index 4, so it is swapped with index 0. The same logic repeats on the remaining unsorted sub-array for each subsequent pass.

4. Total Number of Comparisons

Comparisons per pass: $4 + 3 + 2 + 1 = 10$ comparisons.

This matches the formula for Selection Sort: $\text{comparisons} = n(n-1)/2 = 5 \times 4 / 2 = 10$, confirming the manual count.

Swaps performed: 3 (Pass 1, Pass 2, and Pass 3 each required a swap; Pass 4 needed none since 25 was already in the correct place).

5. Time Complexity Analysis

Case	Complexity	Reasoning
Best Case	$O(n^2)$	Even if the array is already sorted, Selection Sort still scans the entire unsorted segment on every pass to confirm the minimum — comparisons are unaffected by initial order.
Average Case	$O(n^2)$	For a randomly ordered array, the same $n(n-1)/2$ comparisons are always performed.
Worst Case	$O(n^2)$	Even for a reverse-sorted array, the comparison count remains $n(n-1)/2$; only the number of swaps changes.

Space Complexity: $O(1)$, since sorting is done in place with only a constant amount of extra memory (for tracking the minimum index).

6. Verification of Correctness

Final output: [11, 12, 22, 25, 64]. Checking pairwise: $11 \leq 12 \leq 22 \leq 25 \leq 64$ — every element is less than or equal to the one that follows it, and the multiset of values matches the original input exactly (no element added, lost, or duplicated). The output is therefore verified as a correct ascending sort.

7. Interpretation of Efficiency

Selection Sort guarantees $O(n^2)$ comparisons in every case, making it inefficient for large datasets compared to $O(n \log n)$ algorithms such as Merge Sort or Quick Sort. However, its key strength is that it performs at most $n-1$ swaps in total, which makes it attractive in situations where write/swap operations are expensive relative to comparisons (for example, sorting data in flash memory). For the small dataset of 5 elements used here, the quadratic overhead is negligible, and the algorithm's simplicity and predictability make it a reasonable brute-force choice for demonstration and for small-scale sorting tasks.

Q2. Closest Pair — Brute Force Analysis

1. Problem Interpretation

Given six points $\{(2,3), (12,30), (40,50), (5,1), (12,10), (3,4)\}$, the task is to find the pair of points with the smallest Euclidean distance between them. This is the classic Closest Pair of Points problem, common in computational geometry.

2. Why the Brute Force Method is Justified

- With only $n = 6$ points, the exhaustive approach of checking every possible pair is simple, guaranteed correct, and computationally cheap — no need for the added complexity of a divide-and-conquer solution.
- Brute Force directly reflects the problem definition: the closest pair must be one of the $C(n,2)$ possible pairs, so evaluating all of them is a natural and reliable method for small n .
- It serves as a correctness baseline against which faster $O(n \log n)$ divide-and-conquer algorithms can be validated.

3. Distance Calculations for All Pairs

Using the Euclidean distance formula $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$, all $C(6,2) = 15$ pairs are evaluated:

Pair	Δx	Δy	Distance (exact)	Distance ($\approx$)
(2,3)–(12,30)	10	27	$\sqrt{829}$	28.79
(2,3)–(40,50)	38	47	$\sqrt{3653}$	60.44
(2,3)–(5,1)	3	2	$\sqrt{13}$	3.61
(2,3)–(12,10)	10	7	$\sqrt{149}$	12.21
(2,3)–(3,4)	1	1	$\sqrt{2}$	1.41
(12,30)–(40,50)	28	20	$\sqrt{1184}$	34.41
(12,30)–(5,1)	7	29	$\sqrt{890}$	29.83
(12,30)–(12,10)	0	20	$\sqrt{400}$	20.00
(12,30)–(3,4)	9	26	$\sqrt{757}$	27.51
(40,50)–(5,1)	35	49	$\sqrt{3626}$	60.22
(40,50)–(12,10)	28	40	$\sqrt{2384}$	48.83
(40,50)–(3,4)	37	46	$\sqrt{3485}$	59.03
(5,1)–(12,10)	7	9	$\sqrt{130}$	11.40
(5,1)–(3,4)	2	3	$\sqrt{13}$	3.61
(12,10)–(3,4)	9	6	$\sqrt{117}$	10.82

4. Identifying the Closest Pair

Minimum distance: $\sqrt{2} \approx 1.41$, occurring between points (2,3) and (3,4).

All other 14 distances are larger than 3.6, so this pair is unambiguously the closest.

5. Comparison Count

Distance computations: $C(6,2) = 15$.

Comparisons to determine the minimum: 14 (comparing each new distance against the current minimum while scanning the list of 15 values).

Total operations: 15 computations + 14 comparisons = 29.

6. Time Complexity Analysis

For n points, the number of pairs is $C(n,2) = n(n-1)/2$, so the Brute Force Closest Pair algorithm runs in $O(n^2)$ time and $O(1)$ extra space. This is efficient for small n but becomes impractical for very large datasets, where the divide-and-conquer approach achieving $O(n \log n)$ is preferred.

7. Verification of Correctness

Re-checking (2,3) and (3,4): $\Delta x = 1$, $\Delta y = 1$, distance = $\sqrt{1^2+1^2} = \sqrt{2} \approx 1.414$. Comparing this against every other computed distance in the table confirms it is the smallest value, so the result is verified as correct.

8. Real-World Applicability

The Closest Pair problem models many practical scenarios: finding the nearest facility (hospital, ATM, warehouse) to a location, air-traffic or ship collision-risk detection, clustering algorithms in machine learning, and VLSI circuit design where components placed too close together can cause interference. Brute Force is suitable when the number of points is small (as here); for large-scale systems (e.g., national GPS networks), the more scalable divide-and-conquer or spatial-indexing methods are used instead.

Q3. Convex Hull — Brute Force Approach

1. Problem Interpretation and Concept of Convex Hull

Given the points $\{(0,3), (2,2), (1,1), (2,1), (3,0), (0,0), (3,3)\}$, we must find the Convex Hull — the smallest convex polygon that encloses all the given points. Intuitively, if a rubber band were stretched around all the points and released, the shape it forms is the convex hull; only the outermost 'boundary' points become hull vertices, while interior points are excluded.

2. Brute Force Logic

The Brute Force Convex Hull algorithm checks every possible pair of points (P_i, P_j) and treats the line through them as a candidate hull edge. For each candidate line, it examines all remaining points: if every other point lies strictly on one side of the line (or on the line itself), then that line segment is a valid edge of the convex hull. If points exist on both sides, the segment is rejected.

3. Checking All Line Pairs

With $n = 7$ points, the total number of point pairs (candidate lines) is $C(7,2) = 21$. Each candidate line is tested against the remaining 5 points, giving $21 \times 5 = 105$ side-checks (comparisons) in total.

4. Determining Valid Hull Edges

Plotting the points shows four points forming the outer corners of a square — $(0,0), (3,0), (3,3), (0,3)$ — while $(2,2), (1,1)$, and $(2,1)$ fall strictly inside this square.

Candidate Edge	Test Result	Valid Hull Edge?
$(0,0)-(3,0)$	All remaining points have $y \geq 0$	Yes
$(3,0)-(3,3)$	All remaining points have $x \leq 3$	Yes
$(3,3)-(0,3)$	All remaining points have $y \leq 3$	Yes
$(0,3)-(0,0)$	All remaining points have $x \geq 0$	Yes
$(0,0)-(3,3)$	Points lie on both sides (e.g., $(0,3)$ above, $(3,0)$ below the line $y=x$)	No
$(2,2)-(1,1)$	Other points lie on both sides	No
(others)	Fail the same one-sided test	No

5. Logical Steps to Determine the Boundary

- Step 1: List all 21 point pairs as candidate edges.
- Step 2: For each candidate, compute the cross product sign of every other point relative to the line.
- Step 3: Keep the edge only if all cross-product signs are the same (all non-negative or all non-positive).
- Step 4: Collect the accepted edges and order them to trace the polygon boundary: $(0,0) \rightarrow (3,0) \rightarrow (3,3) \rightarrow (0,3) \rightarrow$ back to $(0,0)$.

6. Comparisons Performed

Total comparisons: 21 candidate pairs $\times$ 5 remaining points each = 105 side-of-line checks.

7. Time Complexity Analysis

For n points, there are $O(n^2)$ candidate pairs, and each is checked against the remaining $n-2$ points, giving a total time complexity of $O(n^3)$. This is far less efficient than the $O(n \log n)$ Graham Scan or Divide-and-Conquer convex hull algorithms, but the brute force method is easy to reason about and correct by construction, making it suitable for small point sets or for validating faster implementations.

8. Interpretation and Practical Significance

The resulting convex hull is the square with vertices $(0,0)$, $(3,0)$, $(3,3)$, $(0,3)$; the interior points $(1,1)$, $(2,1)$, $(2,2)$ do not affect the boundary. Convex hull computation is widely used in image processing and pattern recognition (shape outlining), geographic information systems (region boundaries), robotics (collision/path planning), and computer graphics. Recognizing which points are 'redundant' (interior) versus which define the true boundary is valuable whenever a minimal enclosing shape is needed for further computation.

Q4. Searching Techniques — Comparative Analysis

1. Problem Interpretation

The task is to analyze and compare Linear Search (applicable to unsorted data) and Binary Search (applicable only to sorted data), in terms of their working, comparison counts, and time complexity, and to determine which is preferable under different conditions.

2. Working of Linear Search

Linear Search scans the array sequentially from the first element, comparing each element with the target value, until a match is found or the array is exhausted. It requires no ordering of the data.

3. Working of Binary Search

Binary Search operates only on sorted data. It repeatedly compares the target with the middle element of the current search range: if equal, the search ends; if the target is smaller, the search continues in the left half; if larger, it continues in the right half. This halves the search space at every step.

4. Applying Both Methods — Worked Example

Linear Search: Unsorted array [64, 25, 12, 22, 11, 90, 45], target = 22.

Step	Index Checked	Value	Match?
1	0	64	No
2	1	25	No
3	2	12	No
4	3	22	Yes — Found

Comparisons required: 4.

Binary Search: Sorted array [11, 12, 22, 25, 45, 64, 90], target = 22.

Step	Low	High	Mid	Value at Mid	Decision
1	0	6	3	25	$25 > 22 \rightarrow$ search left half
2	0	2	1	12	$12 < 22 \rightarrow$ search right half
3	2	2	2	22	Match — Found

Comparisons required: 3.

5. Time Complexity Analysis

Algorithm	Best Case	Average Case	Worst Case
Linear Search	$O(1)$ — target is first element	$O(n)$	$O(n)$ — target is last, or absent
Binary Search	$O(1)$ — target is the middle element	$O(\log n)$	$O(\log n)$

6. Comparative Efficiency Based on Dataset Conditions

- On unsorted data: only Linear Search is directly applicable; Binary Search would require sorting first ($O(n \log n)$ overhead).

- On sorted, large datasets: Binary Search is dramatically faster — e.g., for $n = 1,000,000$, Linear Search may need up to 1,000,000 comparisons, whereas Binary Search needs at most $\lceil \log_2(1,000,000) \rceil \approx 20$.
- On small datasets: the difference is negligible, and Linear Search's simplicity (no sorting needed) can make it preferable.
- For data that changes frequently (frequent insertions/deletions): maintaining sorted order for Binary Search adds overhead, so Linear Search may be more practical.

7. Justification — When Each Method is Preferable

Prefer Linear Search when: the data is unsorted, the dataset is small, or the data changes too often to justify keeping it sorted, or a search is performed only once (a single search doesn't justify $O(n \log n)$ sorting cost).

Prefer Binary Search when: the data is already sorted (or sorted once and searched many times), and the dataset is large enough that $O(\log n)$ offers a substantial advantage over $O(n)$.

8. Real-World Applications

Linear Search is used in scanning small unindexed lists, searching unsorted log files, or simple lookup where implementation simplicity matters more than speed. Binary Search underlies efficient lookups in sorted databases and indices, dictionary and phone-book style lookups, search engine indexing, version-control 'bisect' commands for locating a bug commit, and library functions such as binary search implementations in standard programming language collections.

Q5. Closest Pair — Real-Time Scenario (Emergency Centers)

1. Problem Interpretation

A dispatch system tracks the coordinates of emergency response centers (e.g., ambulance or fire stations) on a city grid, represented here as five centers: E1(4,7), E2(9,10), E3(2,2), E4(13,4), E5(6,6) (units in kilometers on a local grid). The task is to identify which two centers are nearest to each other — useful, for example, for coverage planning or for deciding which center can back up another fastest.

2. Justification of the Chosen Method

- With only $n = 5$ centers, exhaustively checking every pair is fast, simple, and guaranteed to be correct.
- Brute Force avoids the added implementation complexity of divide-and-conquer, which only pays off for much larger n .
- Emergency-planning decisions of this kind are typically recomputed only when centers are added or relocated, so a straightforward and verifiable method is appropriate.

3. Distance Calculations for All Pairs

With $n = 5$ centers, there are $C(5,2) = 10$ possible pairs. Using the Euclidean distance formula:

Pair	Δx	Δy	Distance (exact)	Distance ($\approx$ km)
E1(4,7)–E2(9,10)	5	3	$\sqrt{34}$	5.83
E1(4,7)–E3(2,2)	2	5	$\sqrt{29}$	5.39
E1(4,7)–E4(13,4)	9	3	$\sqrt{90}$	9.49
E1(4,7)–E5(6,6)	2	1	$\sqrt{5}$	2.24
E2(9,10)–E3(2,2)	7	8	$\sqrt{113}$	10.63
E2(9,10)–E4(13,4)	4	6	$\sqrt{52}$	7.21
E2(9,10)–E5(6,6)	3	4	$\sqrt{25}$	5.00
E3(2,2)–E4(13,4)	11	2	$\sqrt{125}$	11.18
E3(2,2)–E5(6,6)	4	4	$\sqrt{32}$	5.66
E4(13,4)–E5(6,6)	7	2	$\sqrt{53}$	7.28

4. Identifying the Nearest Centers

Minimum distance: $\sqrt{5} \approx 2.24$ km, between centers E1(4,7) and E5(6,6).

This is clearly the smallest of the 10 computed distances, with the next-closest pair (E2–E5) at 5.00 km — more than double the minimum.

5. Comparison Count

Distance computations: $C(5,2) = 10$.

Comparisons to find the minimum: 9.

Total operations: $10 + 9 = 19$.

6. Time Complexity Analysis

The Brute Force approach runs in $O(n^2)$ time for n centers — fine for a small, local set of centers as in this example. For a large-scale, real-time national system with thousands of centers, $O(n^2)$ would be too slow to recompute on the fly, and a divide-and-conquer closest-pair algorithm ($O(n \log n)$) or spatial data structures such as k-d trees or Voronoi diagrams would be used instead to support real-time queries.

7. Verification of Correctness

Re-computing directly: $\Delta x = 6 - 4 = 2$, $\Delta y = 6 - 7 = -1$, distance = $\sqrt{2^2 + (-1)^2} = \sqrt{5} \approx 2.236$ km. Comparing against all other nine computed distances confirms this is indeed the minimum, verifying the result.

8. Importance in Real-Time Applications

Rapidly identifying the nearest available emergency center directly affects response time in life-critical situations. Applications include ambulance/fire dispatch optimization, identifying redundant or overlapping coverage zones for resource reallocation, disaster-response planning (positioning relief centers so no two are unnecessarily close while ensuring full area coverage), and ride-sharing or delivery-fleet routing where the nearest available unit must be found quickly. Because real-time systems demand very fast responses, brute force is used for small, local computations, while large-scale deployments rely on spatial indexing to keep query times low as the number of centers grows.

Summary Table

Q	Topic	Complexity	Key Result
1	Selection Sort	$O(n^2)$ all cases	Sorted: [11,12,22,25,64]; 10 comparisons
2	Closest Pair	$O(n^2)$	Closest pair: (2,3) & (3,4), distance $\sqrt{2} \approx 1.41$
3	Convex Hull	$O(n^3)$	Hull: (0,0),(3,0),(3,3),(0,3)
4	Linear vs Binary Search	$O(n)$ vs $O(\log n)$	Binary Search far more efficient on sorted, large data
5	Closest Pair (GPS)	$O(n^2)$	Nearest centers: E1(4,7) & E5(6,6), distance $\sqrt{5} \approx 2.24$